Node.js & Express Architecture Cheat Sheet

NODE V20+ & EXPRESS 4/5

Guía de Referencia Backend de Nivel Senior / Experto

1. Arquitectura Interna de Node.js y Event Loop

Motor Libuv y Fases del Event Loop

- **Single-Threaded Model:** JS corre en un solo hilo (V8), pero las operaciones I/O no bloqueantes se delegan a *Libuv*.
- **Fases del Event Loop:**
  - `Timers`: Ejecuta callbacks de `setTimeout` / `setInterval`.
  - `Pending I/O`: Callbacks de I/O diferidos.
  - `Poll`: Recupera nuevos eventos I/O.
  - `Check`: Ejecuta callbacks de `setImmediate()`.
- **Microtasks Queue:** `process.nextTick()` y `Promises` tienen prioridad absoluta y se vacían entre cada fase.

Concurrencia vs Bloqueo

- **Non-Blocking I/O:** Operaciones de disco o red deben ser asíncronas para no congelar el Event Loop.
- **ThreadPool (Libuv):** Por defecto maneja 4 hilos para tareas pesadas (FS, Crypto, DNS lookup). Ajustable con `UV_THREADPOOL_SIZE`.
- **Worker Threads:** Módulo nativo (`worker_threads`) para procesar tareas CPU-intensive sin bloquear el hilo principal.

2. Express.js: Cadena de Middlewares y Manejo de Errores

Patrón Middleware (Pipeline)

- Un middleware es una función con firma `(req, res, next)`. Si no llama a `next()` ni responde, la petición se "cuelga".
- **Tipos de Middleware:** De Aplicación (`app.use`), De Ruta, Integrados (`express.json()`) y de Terceros (`cors`, `helmet`).
- **Routing Modular:** Uso de `express.Router()` para desacoplar endpoints por dominio o versión (API v1/ v2).

Manejo de Errores Global

- **Middleware de Error:** Requiere 4 parámetros obligatorios: `(err, req, res, next)`.
- **Async Errors:** En Express 4, los errores dentro de promesas deben pasarse con `next(err)` o usar wrappers/Express 5.
- **Seguridad Esencial:** Uso de `helmet` (cabeceras HTTP), `express-rate-limit` y saneamiento de entradas contra inyecciones.

3. Rendimiento, Escala y Patrones de Producción

Diferenciador Senior / Producción: Nunca confíes el escalado vertical a un solo hilo. Implementa el módulo nativo `cluster` o un gestor como `PM2` para correr una instancia por núcleo CPU, y maneja siempre eventos de terminación limpia (*Graceful Shutdown* con `SIGTERM` / `SIGINT`).

Manejo de Memoria y Streams

- **Streams & Buffers:** Procesar grandes volúmenes de datos en chunks (`stream.pipe()`) evita desbordamientos de RAM (High Water Mark).
- **Leaks de Memoria:** Evitar variables globales, timers sin limpiar y closures duraderas manteniendo referencias a objetos grandes.

Buenas Prácticas de Arquitectura

- **Estructura por Capas:** Routes → Controllers → Services (Lógica de Negocio) → Data Access (ORMs/ODMs).
- **Variables de Entorno:** Validación estricta con bibliotecas como `zod` o `joi` al arrancar la app.

4. Servidor Express Producción-Ready (ES Modules / Async Clean Architecture)

```
import express from 'express';
import helmet from 'helmet';

const app = express();

// 1. Middlewares de Seguridad y Parsing
app.use(helmet());
app.use(express.json());

// 2. Wrapper para Aislamiento de Errores Asíncronos
const asyncHandler = (fn) => (req, res, next) =>
  Promise.resolve(fn(req, res, next)).catch(next);

// 3. Endpoint con Lógica Asíncrona
app.get('/api/v1/users/:id', asyncHandler(async (req, res) => {
  const { id } = req.params;
  if (!id || isNaN(Number(id))) {
    const err = new Error('ID de usuario inválido');
    err.status = 400;
    throw err;
  }
  // Simulación de consulta a BD
  res.status(200).json({ success: true, data: { id, role: 'Admin' } });
}));

// 4. Middleware Global de Manejo de Errores (4 argumentos obligatorios)
app.use((err, req, res, next) => {
  const statusCode = err.status || 500;
  console.error(`[ERROR] ${err.message}`);
  res.status(statusCode).json({
    error: { message: err.message || 'Error Interno del Servidor' }
  });
});

// 5. Graceful Shutdown
const server = app.listen(3000, () => console.log('Server en puerto 3000'));
process.on('SIGTERM', () => server.close(() => process.exit(0)));
```